Requirements traceability

Maps every item of the assignment brief to the code that implements it, the evidence that checks it, and an honest status. Version 0.8.1. See architecture.md for the design and testing.md for the test approach. A test checks that every file named here exists.

Status key: Met: implemented and tested. **Met, with limits**: implemented and tested, with a stated limitation. **Partial**: some of it. **Pending**: not done yet.

1. The eight core requirements

#	Requirement (from the brief)	How it is met	Code	Evidence	Status
1	**Requirement understanding**: interpret intent, identify ambiguity, normalise into an engineering problem	A `requirements` stage turns free text into goal, functional items, **ambiguities** and acceptance criteria, and a gate rejects missing parts. An ambiguous request goes to a `clarifications` stage where a human answers the high-risk questions	`orchestrator/workflows/shortener.py`, `orchestrator/workflows/ambiguous.py`, `orchestrator/governance/policies/clarification.py`, `orchestrator/llm/contracts.py`	`tests/test_clarification.py`, `tests/test_ambiguous.py`, scenario 03, ADR 005	**Met, with limits**: with a real model, quality depends on the model
2	**Task decomposition**: actionable tasks with dependencies and sequencing	A `plan` stage produces tasks with dependencies, ordering and traceability to acceptance criteria; five planning policies check it is well-formed, maps to the real workflow, sequences consistently with the graph, and covers every criterion	`orchestrator/governance/policies/planning.py`, `orchestrator/workflows/plan_fixtures.py`, `orchestrator/workflows/reference/shortener/plan.json`	`tests/test_planning.py`, scenario 01	**Met**
3	**Codebase reasoning (brownfield)**: impacted modules, APIs, data flows	An `impact_analysis` stage reads the existing code with AST tools (symbols, call graph, public API) and declares the blast radius; policies check it is grounded in real symbols and that the patch stays inside it and keeps the public API	`orchestrator/governance/codeintel.py`, `orchestrator/workflows/brownfield.py`, `orchestrator/governance/policies/compliance.py`, `orchestrator/governance/policies/change_control.py`	`tests/test_codeintel.py`, `tests/test_brownfield.py`, scenario 02, ADR 003	**Met, with limits**: Python AST analysis only
4	**Workflow orchestration (critical differentiator)**	See section 2 for each phrase of the requirement	`orchestrator/core/`, `orchestrator/governance/`, `orchestrator/observability/`	section 2	**Met** (limits noted below)

#	Requirement (from the brief)	How it is met	Code	Evidence	Status
5	**Engineering output generation**: production-quality code, API/schema definitions, unit and integration tests, documentation	A complete shortener (create, redirect, analytics, aliases, expiry, rate limiting, delete) in the standard library with SQLite; a web UI; design, user documentation and tests. The pipeline gates the generated code, tests (they must pass against the code) and documentation (must cover the public API)	`orchestrator/workflows/reference/shortener/shortener.py.txt`, `orchestrator/workflows/reference/shortener/tests.py.txt`, `orchestrator/workflows/reference/shortener/docs.md`, `orchestrator/workflows/reference/shortener/design.md.txt`, `orchestrator/workflows/reference/shortener/ui.html.txt`	`tests/test_shortener_service.py`, `tests/test_shortener_workflow.py`, `tests/test_ui.py`, `tests/test_smoke_script.py`, `scripts/smoke_test.py`	**Met, with limits**: the API is documented in prose and tables, with no machine-readable OpenAPI file
6	**Validation and risk control**: risks, trade-offs, failure scenarios, safety guardrails	Named policies, gates, a sandboxed test runner, workflow lint, fault injection for failure scenarios, deliberate "mutation checks" of the guards, and a decision table of trade-offs	`orchestrator/governance/policies/`, `orchestrator/governance/gates.py`, `orchestrator/governance/sandbox.py`	`tests/test_policies.py`, `tests/test_resilience.py`, policies.md, architecture.md sections 6, 11 and 13	**Met**
7	**Controlled autonomy**: agents execute, humans oversee, approve and own final quality	Approvals on design and release (default deny), risk-graded questions where only a person answers high-risk ones, a safe-stop, and a local admin page where all of this happens	`orchestrator/governance/approvals.py`, `orchestrator/core/engine.py`, `orchestrator/admin/session.py`	`tests/test_admin_api.py`, `tests/test_admin_ui.py`, admin-page.md, ADR 007	**Met**
8	**Final engineering summary**: plan and rationale, artifacts, risks and trade-offs and validation, assumptions, limitations	One document with the plan and rationale, the artifacts, validation evidence generated by running the scenarios and the scripted faults, a risk register, assumptions, limitations and next steps, and an open account of how AI was used	`docs/final-summary.md`	`tests/test_docs.py`, ai-usage-log.md, architecture-reference.md	**Met**

2. Requirement 4 in detail: the orchestration layer

Phrase in the brief	How it is met	Code	Evidence	Status
Coordinates the full lifecycle: requirements, design, implementation, testing, documentation, release readiness	The greenfield workflow has all of these stages plus plan, UI and review	`orchestrator/workflows/shortener.py`	`tests/test_shortener_workflow.py`, generated graph in architecture.md	**Met**

Phrase in the brief	How it is met	Code	Evidence	Status
Non-linear, stateful, governed; not simple chaining	A dependency graph executed in parallel waves with node and run states, checkpoints and rollback	`orchestrator/core/engine.py`, `orchestrator/core/state.py`	`tests/test_core.py`, `tests/test_resilience.py`	**Met**
Explicit dependency graph with entry and exit gates	`Graph` and `Node` with entry gates, exit gates, tags; validated acyclic; workflow lint refuses a graph missing mandatory controls	`orchestrator/core/graph.py`, `orchestrator/governance/gates.py`, `orchestrator/governance/policies/change_control.py`	`tests/test_core.py`, `tests/test_policies.py`, every stage's controls in architecture-reference.md	**Met**
Sequential and parallel paths with synchronisation	Ready nodes run in parallel; a barrier ends each wave. In the greenfield graph, tests, docs and UI run in parallel and join at review	`orchestrator/core/engine.py`	`tests/test_core.py`	**Met, with limits**: wave-level barrier, so a slow node delays siblings' successors
Preserve cross-stage context and decision lineage	Agents read a read-only snapshot of committed outputs; outputs record what they derived from and which agent produced them; revisions record who, why and fingerprints; the report has a lineage section	`orchestrator/core/engine.py`, `orchestrator/runner.py`	`tests/test_replan.py`, `tests/test_clarification.py`	**Met**
Human approval checkpoints for high-impact actions	`requires_approval` nodes (design, release); workflow lint requires it for anything tagged high-impact; approvals are made on the CLI or the admin page	`orchestrator/core/engine.py`, `orchestrator/governance/approvals.py`	`tests/test_core.py`, `tests/test_admin_api.py`	**Met**
Bounded retries, fallback, rollback, safe-stop	The resilience ladder with a global attempt budget; rejection fed back to the agent; checkpoint and whole-change rollback; a reviewer's rejection is sent back for a bounded rework	`orchestrator/core/engine.py`	`tests/test_resilience.py`, `tests/test_rework.py`, ADR 001, ADR 009	**Met**
Policy guardrails for security, compliance and change control	39 named controls in three categories, each reporting `[category/id]` when it blocks	`orchestrator/governance/policies/security.py`, `orchestrator/governance/policies/compliance.py`, `orchestrator/governance/policies/change_control.py`	`tests/test_policies.py`, policies.md, ADR 002	**Met, with limits**: static analysis and pattern checks, not a full SAST tool

Phrase in the brief	How it is met	Code	Evidence	Status
Audit-grade observability and traceability	A hash-chained audit log of 24 event types, verifiable on the admin page; a run report	`orchestrator/observability/audit_log.py`, `orchestrator/runner.py`	`tests/test_core.py`, `tests/test_admin_api.py`	**Met, with limits**: tamper-evident, not tamper-proof without external anchoring
Reliability metrics: success rate, retry and rollback frequency, MTTR, end-to-end latency	All derived from the audit events, defined in the metrics module and in the architecture overview	`orchestrator/observability/metrics.py`	`tests/test_core.py`, `tests/test_replan.py`	**Met**
Dynamically re-plan when upstream outputs change	Content fingerprints identify stale nodes; `revise()` applies a change and redoes only what depends on it; identical edits do nothing	`orchestrator/core/engine.py`	`tests/test_replan.py`, `tests/test_ambiguous.py`, ADR 004	**Met, with limits**: triggered by a revision made through the engine (a person or an agent), not by watching external systems; tracking all ancestors is conservative
...while maintaining governance and controlled autonomy	A revision needs a reason, respects the re-plan budget, must pass the node's own gates, and is rolled back as a whole if the redo fails	`orchestrator/core/engine.py`	`tests/test_replan.py`, `tests/test_clarification.py`	**Met**

3. Deliverables

Deliverable	Where	Status
Working prototype, runnable end to end	`README.md` (setup and commands), `orchestrator/cli.py`, `orchestrator/admin/server.py`, `scripts/smoke_test.py`	**Met**
Architecture overview (components, orchestration model, control flow, key decisions)	architecture.md (also as a PDF), architecture-reference.md	**Met**
Three scenarios (greenfield, brownfield, ambiguous), each showing decomposition, orchestration and validation	scenario 01, scenario 02, scenario 03	**Met**

Deliverable	Where	Status
Setup instructions	`README.md`	**Met, with limits**: a check on a clean machine is still to be done
Testing approach, limitations, trade-offs	testing.md, final-summary.md, architecture.md sections 11 and 13	**Met**

4. Evaluation criteria

Criterion	Where to look
Effectiveness of agentic orchestration	Section 2 above; architecture.md sections 4 and 5; `tests/test_core.py`, `tests/test_resilience.py`, `tests/test_replan.py`
Architecture and system design quality	architecture.md (layers, injected dependencies, trust boundaries); decisions/
Depth of decomposition and execution quality	the `plan` stage and its policies; the three scenarios
Realism and quality of outputs	`orchestrator/workflows/reference/shortener/shortener.py.txt` and its tests; the real-model path in real-llm.md (tested against a fake API, plus one complete live run by the author)
Validation and risk management rigour	policies.md; `tests/test_policies.py`; mutation checks recorded in ai-usage-log.md; testing.md
Clarity and defensibility of decisions	the decision table in architecture.md section 11 and the nine ADRs in decisions/
Modular, testable, reliable, secure, scalable code with safe change management	injected dependencies; over 350 tests; the resilience ladder; named policies and trust boundaries; change control and re-plan rollback. **Scalable:** see the limits below
Engineering judgement	the trade-offs and limits in architecture.md; three real-world (Windows) bugs found, fixed and recorded in testing.md; an honest AI usage log

5. Known gaps, stated plainly

- **The real-model path has had one complete live run, not a measurement.** The author's first live runs (Sonnet) exposed four gaps, now fixed, and a full run has since completed. It is otherwise tested through a fake server with the same wire format; expect prompt tuning.
- **No machine-readable API definition** (OpenAPI); the API and schema are documented in prose and tables and in the code.
- **Scalability is a prototype's:** one machine, SQLite, wave scheduling, run state in memory (no crash-resume). The design keeps these behind clear boundaries, and architecture.md section 13 lists the next steps.
- **Static analysis only** for security and compliance checks.
- **A clean-machine setup check** and load or cross-browser testing have not been done.

AI assistance was used throughout, as the brief invites; what was generated, what was checked and what was found wrong is recorded in ai-usage-log.md.